

ELRS Ground Station

Benutzerhandbuch

Ein schlanker Ground-Control-Bildschirm fuer
ExpressLRS (ELRS) Modelle

Stand: August 2026

github.com/KresserSimon/ELRS_Telemetry_Groundcontrol

Inhalt

- 1. Einleitung
- 2. Installation
- 3. Erste Schritte
- 4. Verbindung zur Telemetrie herstellen
- 5. Die Benutzeroberfläche
- 6. Die Menues im Detail
- 7. Route/Wegpunkte planen und importieren
- 8. Fluglog (CSV-Aufzeichnung)
- 9. Akkuwarnung: LiPo vs. Li-Ion
- 10. Als eigenständige .exe kompilieren
- 11. Anhang: Kommandozeilen-Referenz

1. Einleitung

ELRS Ground Station ist eine leichtgewichtige Alternative zu Mission Planner oder QGroundControl fuer Modelle mit ExpressLRS (ELRS). Die App zeigt live auf einer OpenStreetMap-Karte, wo sich das Modell befindet, stellt alle wichtigen Telemetriedaten (GPS, Akku, Funkverbindung, Sensoren) in einem uebersichtlichen Dashboard dar, warnt per Sprachausgabe bei niedrigem Akkustand und erlaubt es, Flugpfade aufzuzeichnen, zu exportieren und Routen/Wegpunkte zu planen.

Die App funktioniert mit zwei Telemetrie-Protokollen:

- MAVLink - ausgegeben von Flugsteuerungen wie ArduPilot, iNav oder Betaflight, die per CRSF-Telemetrie mit dem ELRS-Empfänger verbunden sind.
- CRSF (Crossfire) - das native Telemetrieprotokoll von ExpressLRS. CRSF wurde urspruenglich von TBS (Team BlackSheep) fuer deren Crossfire-Funksystem entwickelt; ExpressLRS verwendet bewusst dasselbe Frameformat, sodass die App auch mit echter TBS-Crossfire-Hardware funktioniert.

Beide Protokolle lassen sich sowohl ueber WiFi (UDP) als auch ueber eine direkte USB/seriell-Verbindung empfangen, umschaltbar zur Laufzeit ohne Neustart der App.

2. Installation

Voraussetzung ist Python 3.10 oder neuer. Im Projektordner:

```
cd elrs_ground_station
python -m venv .env
.venv\Scripts\activate
pip install -r requirements.txt
```

Die requirements.txt installiert PyQt6 (inkl. WebEngine fuer die Karte und WebChannel fuer die Routen-Interaktion), pymavlink, pytsx3 (Sprachausgabe) und pyserial (USB-Verbindungen). Unter Windows nutzt pytsx3 die eingebaute SAPI5-Sprachausgabe - es sind keine zusaetzlichen Systempakete noetig.

Alternativ steht eine fertig kompilierte .exe zur Verfuegung (siehe Abschnitt 10), die ohne Python-Installation laeuft.

3. Erste Schritte

Zum unverbindlichen Ausprobieren ohne jegliche Hardware:

```
python main.py --demo
```

Der Demo-Modus simuliert einen Loiter-Kreisflug inklusive Akku-Entladung, Roll/Pitch-Bewegung, wechselnden Flugmodi und schwankender Funkverbindung, sodass sich alle Funktionen der App - einschliesslich der Sprachwarnung bei niedrigem Akku - ohne echtes Modell testen lassen.

Wird die App ohne --demo gestartet, oeffnet sich zunaechst ein Popup zur Auswahl von Verbindung (WiFi/UDP oder USB) und Protokoll (MAVLink oder CRSF). Ein Klick auf Abbrechen uebernimmt einfach die per Kommandozeile uebergebenen bzw. die Standard-Einstellungen; ein eigener Button im Popup startet stattdessen direkt den Demo-Modus.

4. Verbindung zur Telemetrie herstellen

ELRS-Hardware spricht kein natives 'Telemetrie-ueber-WiFi' - das eingebaute WiFi eines ELRS-Moduls dient primaer dem Flashen/Konfigurieren (Access Point ExpressLRS TX/ExpressLRS RX, Standardpasswort expresslrs). Um Telemetrie tatsaechlich zu dieser App zu bekommen, gibt es drei Wege:

Weg 1: MAVLink ueber WiFi (empfohlen)

Voraussetzung ist eine Flugsteuerung mit MAVLink-Ausgabe, die per CRSF-Telemetrie mit dem ELRS-Empfaenger verbunden ist. Der serielle MAVLink-Strom der Flugsteuerung muss per WiFi-Bridge (z. B. ein ESP32/ESP8266 mit MAVESP8266-Firmware) auf UDP-Port 14550 gebracht werden.

```
python main.py --protocol mavlink --host 0.0.0.0 --port 14550
```

Muss die Bridge stattdessen aktiv eine Verbindung zum PC aufbauen: --udp-mode connect --host <IP-der-Bridge>

Weg 2: Rohes CRSF/TBS-Crossfire ueber WiFi

Manche ELRS-'Backpack'-Bridges (ESP32-basiert) leiten den rohen CRSF-Bytestrom direkt per UDP weiter, ohne Umweg ueber MAVLink.

```
python main.py --protocol crsf --host 0.0.0.0 --port 14551
```

Dieser Modus deckt GPS (inkl. Geschwindigkeit), Akku (Spannung/Strom/Restkapazitaet/verbrauchte mAh), Link-Statistiken, Attitude (Roll/Pitch) sowie - falls gesendet - Vario, Baro-Hoehe, RPM, Temperatur und Zellspannungen ab.

Weg 3: USB/seriell

Flugsteuerung oder ELRS TX-Modul per USB-Kabel direkt an den PC anschliessen. Verfuegbare Ports zunaechst auflisten:

```
python main.py --list-ports
python main.py --connection usb --protocol mavlink --serial-port COM5
python main.py --connection usb --protocol crsf --serial-port COM5 --baud 420000
```

Standard-Baudrate: 57600 fuer MAVLink, 420000 fuer CRSF (jeweils per --baud ueberschreibbar). Diese Verbindungsart ersetzt Weg 1/2, ist kein Zusatz dazu.

In allen WiFi-Faellen muessen PC und Bridge/Modul im selben Netzwerk sein. Alle drei Wege lassen sich auch nachtraeglich ueber Einstellungen -> Verbindung... aendern, ohne die App neu zu starten.

5. Die Benutzeroberflaeche

5.1 Die Karte

Zeigt die Live-Position des Modells auf einer OpenStreetMap-Karte, mit:

- Nachgezogenem, orangefarbenem Flugpfad seit Verbindungsbeginn.
- Einem Haeuschen-Symbol an der Home-Position (dem ersten empfangenen GPS-Fix der laufenden Sitzung).
- Einem waehlbaren Fahrzeugsymbol (Quadrocopter, Wing, Flugzeug), das sich mit der Kompassrichtung mitdreht.
- Auto-Center: die Karte folgt automatisch der aktuellen Position (abschaltbar).
- Einer optionalen, per Klick gezeichneten oder importierten Route (gruen, gestrichelt, nummerierte Wegpunkte).

5.2 Das Dashboard

Die Leiste am unteren Fensterrand zeigt alle Telemetriedaten gruppiert an:

Gruppe	Felder
GPS	Breitengrad, Laengengrad, Hoehe, Satellitenanzahl
Status	Flugmodus
Link	RSSI, Link-Qualitaet (LQ), SNR, Sendeleistung
Akku	Spannung, Restkapazitaet, Min-Zellspannung, Strom, verbrauchte Kapazitaet (mAh)
Sensoren	Vario, Baro-Hoehe, RPM, Temperatur
Long-Range	Geschwindigkeit, Entfernung/Peilung zur Home-Position, Flugzeit
Verbindung	Status-LED + Text (Verbunden/Getrennt)

Jedes einzelne Feld - nicht nur ganze Gruppen - laesst sich ueber Einstellungen -> Dashboard anpassen... ein- oder ausblenden. Die getroffene Auswahl wird als persoenerlicher Standard in `~/elrs_ground_station/dashboard_fields.json` gespeichert und bei jedem weiteren Start automatisch wieder geladen.

5.3 Kuenstlicher Horizont

Ein Overlay auf der Karte zeigt Roll und Pitch als klassischer Fluglageanzeiger, gespeist aus MAVLink- oder CRSF-Attitude-Daten. Das Overlay laesst sich:

- frei verschieben - einfach mit der Maus anklicken und an eine beliebige Stelle auf der Karte ziehen,
- ueber Menue-Presets auf eine feste Ecke springen lassen (oben links/rechts, unten links/rechts),
- in der Groesse skalieren (75%-200%),
- komplett ein-/ausblenden.

Alle Optionen dazu liegen unter Einstellungen -> Ansicht.

6. Die Menues im Detail

6.1 Datei

- Flugpfad als GPX exportieren... / als KML exportieren... - speichert alle bisher aufgezeichneten GPS-Punkte des aktuellen Fluges in eine Datei.

- Beenden

6.2 Route

- Wegpunkt-Modus - schaltet den Klick-zum-Hinzufuegen-Modus auf der Karte ein. Ein Klick auf einen bestehenden Wegpunkt entfernt genau diesen wieder.
- Letzten Wegpunkt entfernen - nimmt den zuletzt hinzugefuegten Punkt wieder zurueck.
- Route loeschen - entfernt alle Wegpunkte.
- Route importieren... - laedt eine Wegpunktliste aus einer Datei (siehe Abschnitt 7).

6.3 Fluglog

- Log-Einstellungen... - waehlt, welche der ueber 20 Telemetriefelder aufgezeichnet werden sollen und in welchem Intervall (0,1 bis 60 Sekunden).
- Logging aktiv - fragt bei Aktivierung einen Zielpfad fuer die CSV-Datei ab und beginnt sofort mit der Aufzeichnung; ein erneuter Klick beendet sie.

6.4 Einstellungen

- Verbindung... - wechselt zur Laufzeit zwischen WiFi/UDP und USB/Seriell sowie zwischen MAVLink und CRSF. Beendet dabei automatisch einen laufenden Demo-Modus.
- Akkuwarnung... - siehe Abschnitt 9.
- Dashboard anpassen... - siehe Abschnitt 5.2.
- Ansicht -> Auto-Center, Aktuelle Position anspringen (Strg+Pos1), Fahrzeugtyp, Kuenstlicher Horizont anzeigen/Position/Groesse.
- Sprache -> Deutsch/English, wechselt die komplette Oberflaeche sofort ohne Neustart.

6.5 Simulation

- Demo-Modus - schaltet zur Laufzeit zwischen echter Telemetrie und simulierten Daten um.

7. Route/Wegpunkte planen und importieren

Neben der live aufgezeichneten Flugspur (orange) kann eine unabhaengige, geplante Route (gruen, gestrichelt, mit nummerierten Wegpunkt-Markern) auf der Karte angezeigt werden - entweder von Hand gezeichnet (Route -> Wegpunkt-Modus) oder importiert aus einer Datei (Route -> Route importieren...). Unterstuetzte Formate:

Format	Beschreibung
.gpx	Liest Routenpunkte (<rte>), ersatzweise Track-Punkte oder einzelne Wegpunkte aus einer Standard-GPX-Datei.
.mission	Das "MW XML"-Missionsformat von mwp/iNav Configurator/ezgui - <missionitem>-Elemente mit Breiten-/Laengengrad in Dezimalgrad und Hoehe in Metern.
.xml	Generischer Fallback: durchsucht beliebige XML-Strukturen nach Attributen oder Kindelementen mit erkennbaren Lat/Lon/Alt/Name-Bezeichnungen.
.csv	Erkennt Spalten wie lat/latitude, lon/longitude, optional alt und name (Gross-/Kleinschreibung egal).

8. Fluglog (CSV-Aufzeichnung)

Das Fluglog zeichnet - unabhaengig vom GPX/KML-Export der reinen Flugspur - saemtliche Telemetriedaten als Zeitreihe in einer CSV-Datei auf: Position, Hoehe, Satelliten, GPS-Fix, Kompassrichtung, Roll/Pitch, Flugmodus, Akku (Spannung/Restkapazitaet/Strom/verbrauchte Kapazitaet/Zellspannungen), Funkverbindung (RSSI/LQ/SNR/Sendeleistung), Sensoren (Vario/Baro-Hoehe/RPM/Temperatur), Geschwindigkeit und Verbindungsstatus.

Ueber Fluglog -> Log-Einstellungen... laesst sich sowohl die Menge der aufgezeichneten Spalten als auch das Aufzeichnungsintervall (0,1-60 s) frei waehlen. Nach Klick auf Fluglog -> Logging aktiv wird ein Speicherort abgefragt; ab dann schreibt die App laufend eine Zeile pro Intervall, bis das Haekchen wieder entfernt wird. Aenderungen an den Log-Einstellungen wirken sich erst auf die naechste gestartete Aufzeichnung aus, um die bereits laufende CSV-Datei nicht durch einen inkonsistenten Spaltenkopf zu beschaedigen.

9. Akkuwarnung: LiPo vs. Li-Ion

Sobald der Akku niedrig oder kritisch niedrig wird, gibt die App eine Sprachwarnung aus (offline, ueber die Windows-SAPI5-Sprachausgabe). Da sich die sichere Entladeschlussspannung von LiPo- und Li-Ion-Zellen deutlich unterscheidet, laesst sich die Chemie unter Einstellungen -> Akkuwarnung... auswaehlen:

Chemie	Warnung (V/Zelle)	Kritisch (V/Zelle)
LiPo	3,6 V	3,5 V
Li-Ion	3,3 V	3,0 V

Die Auswahl der Chemie fuellt automatisch passende Standardwerte vor; Zellenzahl sowie die genauen Warn-/Kritisch-Spannungen lassen sich zusaetzlich frei einstellen. Steht eine echte Zellspannungsmessung zur Verfuegung (CRSF-Cells-Frame oder MAVLink BATTERY_STATUS), verwendet die App die tatsaechliche niedrigste Zellspannung fuer die Warnung - das ist praeziser als eine aus der Gesamtspannung geschaetzte Durchschnittszellspannung, da eine einzelne schwache Zelle die tatsaechliche Belastungsgrenze bestimmt.

10. Als eigenstaendige .exe kompilieren

```
cd elrs_ground_station
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt pyinstaller
pyinstaller --name ELRS_GroundStation --onedir main.py
```

Das Ergebnis liegt unter dist\ELRS_GroundStation\ELRS_GroundStation.exe. Der gesamte Ordner dist\ELRS_GroundStation (Exe plus _internal-Verzeichnis mit den Qt/WebEngine-Ressourcen, ca. 500 MB) muss zusammen weitergegeben werden, nicht nur die .exe-Datei allein.

--onedir statt --onefile wird bewusst verwendet: QtWebEngine braucht einen eigenen Hilfsprozess samt Ressourcendateien, die eine Single-File-Exe bei jedem Start erst in ein Temp-Verzeichnis entpacken muesste - das funktioniert, ist aber langsamer beim Start und fehleranfaelliger.

Die Exe behaelt bewusst die Konsole (kein --windowed), damit --list-ports, --demo usw. weiterhin normal ueber die Kommandozeile nutzbar sind; beim Doppelklick oeffnet sich zusaetzlich ein Konsolenfenster im Hintergrund.

11. Anhang: Kommandozeilen-Referenz

Option	Beschreibung
--demo	Im Simulationsmodus starten, keine Hardware noetig.
--protocol {mavlink,crsf}	Telemetrieprotokoll (Standard: mavlink).
--connection {udp,usb}	Transportweg (Standard: udp).
--host	Bind-Adresse fuer UDP-Empfang (Standard: 0.0.0.0).
--port	UDP-Port (Standard: 14550 MAVLink / 14551 CRSF).
--udp-mode {listen,connect}	listen wartet auf Pakete, connect verbindet aktiv zu Host:Port.
--serial-port	USB/seriell-Port bei --connection usb, z. B. COM5.
--baud	Baudrate bei USB (Standard: 57600 MAVLink / 420000 CRSF).
--list-ports	Verfuegbare USB/seriell-Ports auflisten und beenden.
--cells	Anzahl LiPo/Li-Ion-Zellen fuer die Akku-Warnschwellen.
--low-cell-voltage	Zellspannung fuer die "niedrig"-Warnung.
--critical-cell-voltage	Zellspannung fuer die "kritisch"-Warnung.
--demo-center lat,lon	Mittelpunkt der Demo-Flugbahn.
--lang {de,en}	Startsprache der Oberflaeche.

Vollstaendige Projektdokumentation, Quellcode und Architektur-Details:
github.com/KresserSimon/ELRS_Telemetry_Groundcontrol